

Resampling & Markov Chain Monte Carlo

Dr Lily Asquith (Lily, she/her)





Resampling & MCMC

- Today: Resampling
- Today: Intro to Markov Chain Monte Carlo
- Wednesday: Tuning MCMC.
- Wednesday: Marking Scheme for Software Exercise.

No amount of data is enough

If you have just enough to do what you want to do now, you will do that thing and gain insight.

That insight will give you a new, better idea for another thing, on a subset of the data.

Now you need more data, because the subset is too small.

One way of tackling the chronic data shortage is to generate 'new' samples of data from your existing one. This is called **Resampling**.

Resampling

You toss a coin 1000 times, generating a dataset of size 1000. Then you find out you needed 1100 data points, not 1000. Which of these should you do?

- 1 Find the Maximum Likelihood Estimate for $\hat{p}$, and generate another 100 tosses using $Bernoulli(\hat{p})$.
- 2 Randomly choose one value from your existing dataset and repeat this 100 times.

Resampling

You toss a coin 1000 times, generating a dataset of size 1000. Then you find out you needed 1100 data points, not 1000. Which of these should you do?

- 1 Find the Maximum Likelihood Estimate for $\hat{p}$, and generate another 100 tosses using $Bernoulli(\hat{p})$.
- 2 Randomly choose one value from your existing dataset and repeat this 100 times.

These two options are the same option, phrased differently!

For example:

- 1 If the MLE for $\hat{p} = 0.54$ you will expect to generate 54 new heads with $Bernoulli(0.54)$.
- 2 If the MLE for $\hat{p} = 0.54$, this is because there are 540 heads in the dataset. You will expect to draw 54 heads from your random picks.

Resampling

You toss a coin 1000 times, generating a dataset of size 1000. Then you find out you needed 1100 data points, not 1000. Which of these should you do?

- 1 Find the Maximum Likelihood Estimate for $\hat{p}$, and generate another 100 tosses using $Bernoulli(\hat{p})$. ← Parametric Resampling
- 2 Randomly choose one value from your existing dataset and repeat this 100 times. ← Nonparametric Resampling

These two options are the same option, phrased differently!

For example:

- 1 If the MLE for $\hat{p} = 0.54$ you will expect to generate 54 new heads with $Bernoulli(0.54)$.
- 2 If the MLE for $\hat{p} = 0.54$, this is because there are 540 heads in the dataset. You will expect to draw 54 heads from your random picks.

Resampling: the Bootstrap

One reason you want a larger dataset is to calculate a **Statistic** from the data.
For example the mean, mode, standard deviation, χ^2 - but not min/max.
Dataset has size $N_{dat} = 100$.

- Randomly select a value from the data.
- Do this 100 times, but allow yourself to select the same value more than once (do not replace).
- Calculate the statistic of interest.
- Repeat the above three steps 9999 times.

Why would I want to do this when I already have the sample statistic?

Resampling: the Bootstrap

```
np.random.default_rng()
np.random.seed(123)
import numpy as np
from scipy.stats import norm
sample = norm.rvs(loc=6.0, scale=3.77, size=100)
# Our statistic is the standard deviation
print( f'sample sigma: {np.std(sample):.2f}')
# sample sigma: 4.25
sigmas=[]
for i in range(9999):
    #bs=[ np.random.choice(sample, replace=True) for j in range(100)]
    bs=[]
    for j in range(100):
        a=np.random.choice(sample,replace=True)
        bs.append(a)
    sigmas.append(np.std(bs))
la_cv=f'{np.mean(sigmas):.2f}'
print( f'bootstrap: {la_cv}')
# bootstrap: 4.23
```

Why would I want to do this when I already have the sample statistic?

Resampling: the Bootstrap

```
np.random.default_rng()
np.random.seed(123)
import numpy as np
from scipy.stats import norm
sample = norm.rvs(loc=6.0, scale=3.77, size=100)
# Our statistic is the standard deviation
print( f'sample sigma: {np.std(sample):.2f}')
# sample sigma: 4.25
sigmas=[]
for i in range(9999):
    bs=[ np.random.choice(sample, replace=True) for j in range(100)]
    bs=[]
    for j in range(100):
        a=np.random.choice(sample,replace=True)
        bs.append(a)
    sigmas.append(np.std(bs))
la_ser=f'{np.std(sigmas):.2f}'
la_cv=f'{np.mean(sigmas):.2f}'
print( f'bootstrap: {la_cv}+/-{la_ser}')
# bootstrap: 4.23+/-0.24
```

Because now you have an uncertainty on the sample statistic!

Resampling: the scipy bootstrap

The scipy bootstrap method is a quick, tidy way of doing this, with more features such as a confidence interval (default is 95%, configurable).

```
# Our statistic is the standard deviation
print( f'sample sigma: {np.std(sample):.2f}')
# sample sigma: 4.25
from scipy.stats import bootstrap
bs_result = bootstrap( (sample,), np.std)
sp_ser = f'{bs_result.standard_error:.2f}'
sp_cv = f'{np.mean(bs_result.bootstrap_distribution):.2f}'
print(f'Gorgeous scipy bootstrap: {sp_cv}+/{-}{sp_ser}')
# Gorgeous scipy bootstrap: 4.22+/-0.25
low,high=bs_result.confidence_interval
print( f' 95% CL: {low:.2f}, {high:.2f}')
# 95% CL: 3.83, 4.83
```

Parametric Bootstrap

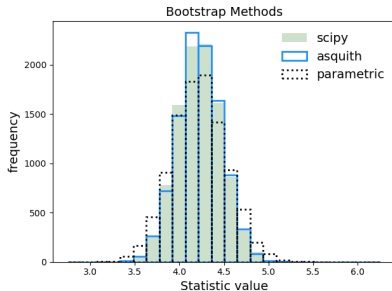
Making additional datasets using the MLE estimate for the parameter (statistic, in this case the standard deviation) is called a Parametric Bootstrap. **You need to know the form of the underlying distribution to use this approach.**

```
# Our statistic is the standard deviation
print( f'sample sigma: {np.std(sample):.2f}')
# sample sigma: 4.25
# Parametrically
para_sigmas=[]
for i in range(9999):
    respin_data = norm.rvs(loc=samp_mean, scale=samp_sigma, size=100)
    para_sigmas.append( np.std(respin_data) )
pr_ser=f'{np.std(para_sigmas):.2f}'
pr_cv=f'{np.mean(para_sigmas):.2f}'
print( f'Slow parametric respin: {pr_cv}+/-{pr_ser}')
#Slow parametric respin: 4.23+/-0.30
```

Comparison

Now we can quote the sample standard deviation as $\sigma_s = 4.23 \pm 0.30$.*

This is a **frequentist process**, so the true value of σ is expected to lie within our error margin 68% of the time.



*This uncertainty was the highest returned by the three methods, so I went with it to be conservative. If I ran again without a seed, a different method would be highest. Random!

Variations on the Bootstrap

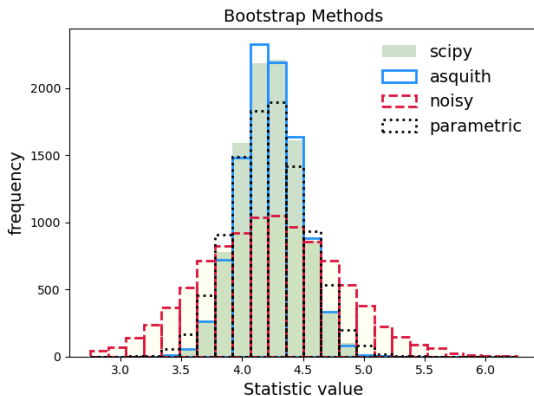
There are uncountable variations on this technique. One handy one is 'smoothing':

```
noisy_sigmas=[]
for i in range(9999):
    bs=[ np.random.choice(sample, replace=True) for j in range(100)]
    noisy_std = norm(np.std(bs),0.5).rvs()
    noisy_sigmas.append( noisy_std )

noisy_ser=f'{np.std(noisy_sigmas):.2f}'
noisy_cv=f'{np.mean(noisy_sigmas):.2f}'
print( f'Noisy bootstrap: {noisy_cv}+/{-}{noisy_ser}')
# Noisy bootstrap: 4.22+/-0.56
# regular bootstrap: 4.23+/-0.25
```

Variations on the Bootstrap

Smoothing: adding noise to the sample inflates the uncertainty but does not alter the mean.



Back to Bayes!

Posterior



Likelihood



$$P(\theta|y)$$

=

$$\frac{\mathcal{L}(y|\theta) \pi(\theta)}{\int \mathcal{L}(y|\theta) \pi(\theta) d\theta}$$

Prior



Evidence

We looked at Poisson-Gamma

Gamma priors: $\pi(\theta) \propto \theta^{\alpha-1} e^{-\beta\theta}$

Poisson likelihoods: $\mathcal{L}(y|\theta) \propto \theta^{n\bar{y}} e^{-n\theta}$

Gamma Posterior: $\text{Gamma}(\alpha + n\bar{y}, \beta + n)$

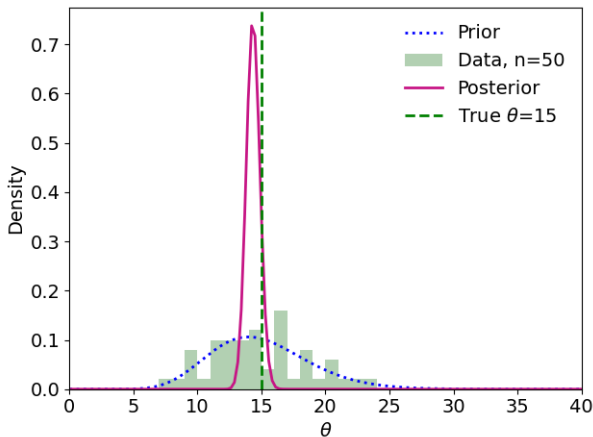
Note that for Gamma, the Expected Value is $E[\theta] = \alpha\beta$ and the Variance is $V[\theta] = \alpha\beta^2$

Compare with Poisson $E[y] = \theta$ and $V[y] = \theta$

For example, if we have Poisson data and guess at the rate parameter $\theta = 15$, we want $\alpha\beta = 15$ and $\alpha\beta^2 = 15$ ie $\alpha = 15, \beta = 1$.

Poisson-Gamma: $\text{Gamma}(\alpha + n\bar{y}, \beta + n) \propto \mathcal{L}(\text{Data}|\theta = 15) \times \text{Gamma}(\alpha, \beta)$

We saw that the analytical Posterior $P(\theta|y)$ does a great job at converging on the mean value of θ , and the MAP estimate $\hat{\theta}$ is the mode of the Posterior.



Poisson-Gamma

Then it was Wednesday evening and we all tried to stay awake while I introduced the Predictive Posterior.

Predictive Posterior

↓

$$P(\tilde{y}|y) = \int_{\Theta} f(\tilde{y}|\theta) P(\theta|y) d\theta$$

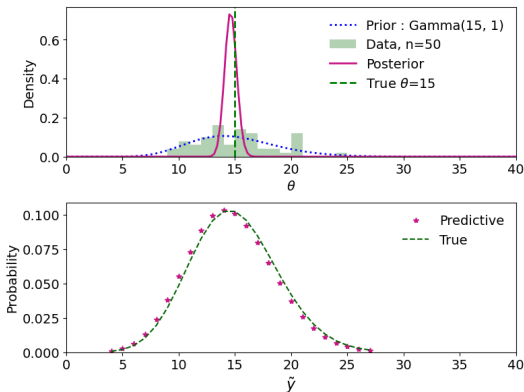
Integrate out θ

↑ PDF or PMF ↑ Posterior

$$P(\theta|y) \propto \mathcal{L}(y|\theta) \pi(\theta)$$

Poisson-Gamma

For Poisson-Gamma the Predictive is a negative binomial. It is awesome because we get the whole distribution as prediction rather.



MCMC

The workshop problem / python example for last week will have showed you the process of building an analytical Predictive posterior.

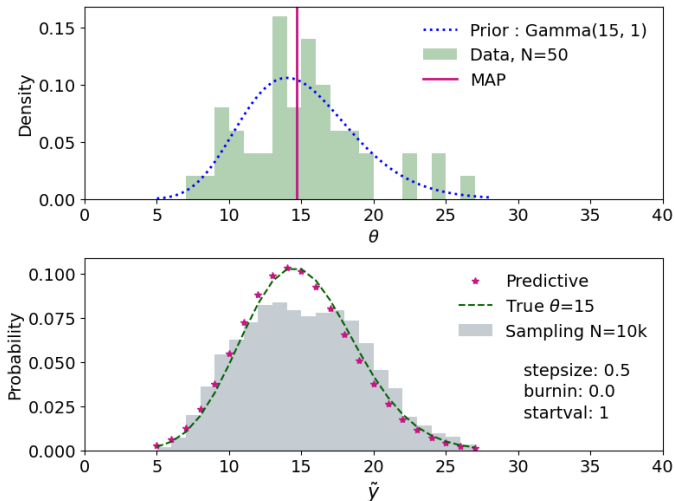
If it is not possible to construct an analytical Predictive Posterior, we can approximate one using MCMC.

To use MCMC we construct the Posterior from the Likelihood and the Prior (which can be uninformative, for example Uniform).

Then we 'fill out' a Sampling distribution. We start at some random parameter value, and take random steps around the parameter space.

The sequence of random steps is called a Markov Chain.

Plot from Poisson-Gamma Example



MCMC

At each step in the Markov Chain we ask:

Is the Posterior for this value $y = \theta_1$ larger than the Posterior for the previous data value $y = \theta_0$?

Yes!: Put this new data value $y = \theta_1$ in a container to include as part of our distribution.

No.: Roll a dice and decide whether to include it or not.

Then we take another step.

The decision at each step depends only on the posterior probability calculated at the previous step, so this sequence gets the privilege of being called a Markov Chain. It is memoryless.

Metropolis Hastings

The example I have provided in `PoissonGamma.py` uses Metropolis Hastings.

One Metropolis Hastings iteration looks like this:

1. Starting value (guess)
2. Add random noise to guess
3. Compare Posteriors, and if the new guess is better, we stick with it. If the new guess is worse, we may still keep it- we assign a probability to it depending on how much worse it is.

Then we repeat. Each iteration is independent of the one before.

What if we have more than one parameter to estimate?

Gibbs

Gibbs sampling is a version of Metropolis that can handle multiple parameters:

1. Start with a pair of guesses for a pair of parameters: a, b .
2. Add random noise to the guess for a .
3. Compare Posteriors, GIVEN b , and if the new guess is better, we stick with it. If the new guess is worse, we may still keep it- we assign a probability to it depending on how much worse it is.

Then repeat this 3-step process for b given a .

This can be super-slow. Differential Evolution is our friend.

Differential Evolution

All the cool kids use Differential Evolution!

Differential Evolution starts with a set of many initial guesses, and generates one chain of samples from each initial guess.

1. Starting value (guess) for chain 1
2. Randomly select a pair of other chains and use the difference in the value between them, Δ (multiplied by a tuning parameter) as the noise. Then add a small amount of random noise (to avoid degeneracy).
3. Usual deal for step 3!

Probabilistic Programming Libraries

PyMC is probably the most widely used. [Docs](#) are a bit opaque, but we live at a fine time for progress on that front.

The emcee library is very easy and is popular with astronomers, but as far as I can tell it is not possible to make Predictive distributions, which seems a bit of a shame. [Docs here](#).

I expect you have instruction in these things if you are going into ML. In case you are not expecting that in your future but want to play with the toys, this looks like an excellent introduction using pymc:

[Bayesian-Methods-for-Hackers](#).

Last lecture Wednesday: Tuning!

Thanks For Your Time

Fin.